gRPC - Remote Procedure Call

Web Services

Jolan PHILIPPE

2026

Université d'Orléans

Table of Contents

Rappel REST

Les principes généraux de RPC

Un peu d'histoire : des RPC à gRPC

gRPC

Rappel REST

API

On veut rendre son application rendre moins monolithique :

- Agilité (développement, et maintenance)
- Sécurité (géo-distribution)
- Performance (scalabilité verticale et horizontale)

Application Programming Interface

Objectif : rendre disponible une ressource par le biais de point d'accès.

REST

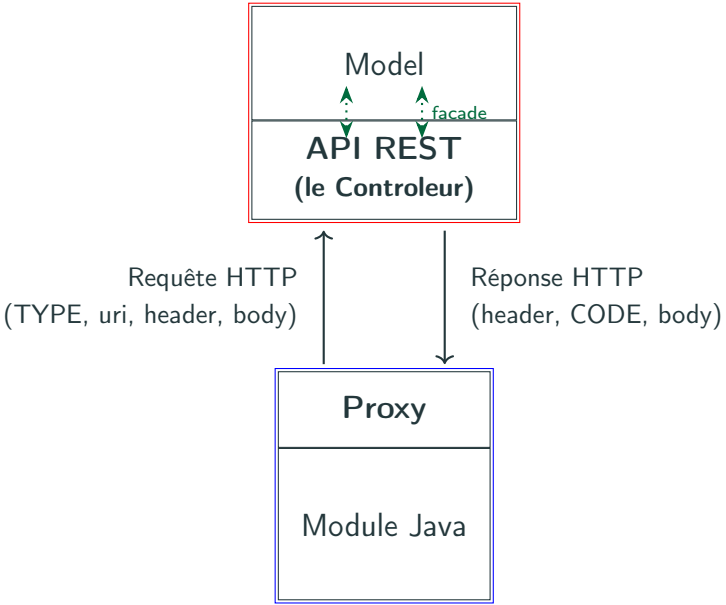
On veut un protocol standard, offrant des points d'entrée simple.

Representational State Transfer

Repose sur le protocol HTTP

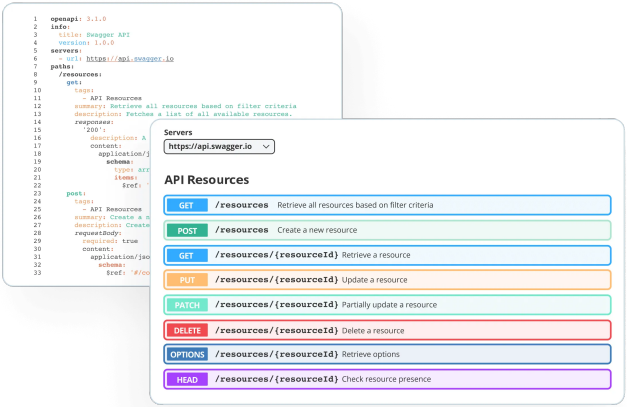
- un verbe, ou méthode, HTTP (par ex. GET, PUT, POST, PATCH, DELETE)
- une uri, ou chemin
- des paramètres sous différentes formes (header, body, path, etc.)

WebService – le “serveur”



Documentation d'une API REST

- Utilisation d'OpenAPI
- Visualisation avec Swagger, SpringDoc, etc.



Limites

- Des URIs ...
- Une absence de controle sur les I/O ...
- Le protocol HTTP ...
- Ecrire le proxy ...

C'est compliqué non ?

Les principes généraux de RPC

RPC

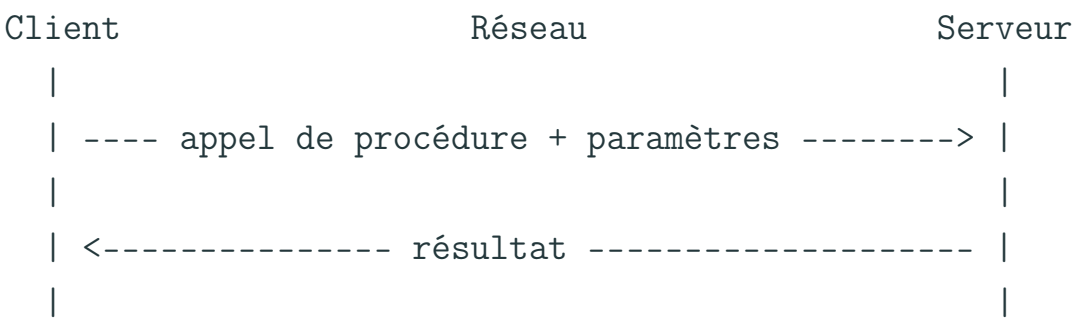
Idée : Abstraire les appels HTTP en appels de fonctions

Remote Procedure Call

- Proposer des fonctions avec des entrées et sorties
- Les types d'entrée et sortie structurés en messages (Messages $\simeq$ DTOs)
- Définir une interface simple, dédiée à l'API, comme une bibliothèque de code
- Utiliser des protocoles standards en interne (HTTP 2.0)

Note pour plus tard : On décrit toute l'API avec un "Interface Definition Language" (IDL)

RPC - Appel de procédure distante



Proxy et “Stub”

- **Coté client** : Une classe proxy intermédiaire, appelée un stub.
- **Coté serveur** : Un stub receptionne les requêtes et appelle le controleur.

A partir d’une spécification (écrite avec l’IDL), les stubs sont du code générés automatiquement.

Roles des Stub

- Sérialisation/désérialisation des entrées / sorties
- (On parle parfois de marshalling/unmarshalling)
- Transformation des données de la mémoire locale pour envoi sur le réseau
- Réception et transformation des données reçues par le réseau
- Gestion de protocoles réseaux utilisés

Tout ça de manière transparente dans le code

Diagramme de fonctionnement



Diagramme de fonctionnement



Diagramme de fonctionnement



Diagramme de fonctionnement



Diagramme de fonctionnement



Interface Definition Language (IDL)

Ecrire la spécification de l'API comme une interface

- Fait office d'un contrat partagé entre le client et le serveur
- Définit la signature des méthodes distantes
- Permet de définir les types des messages d'entrée et de sortie des méthodes

La définition de l'interface est auto-documentante. Pratique.

Note pour plus tard : Dans ce cours, nous utiliserons protobuf

Example

```
syntax = "proto3";

message UserDTO {
  int32 id = 1;
  string name = 2;
  string email = 3;
}

message UserRequest {
  int32 id = 1;
}

service UserService {
  rpc GetUser(UserRequest) returns (UserDTO);
}
```

HTTP/2

Note pour plus tard : Dans ce cours, nous utiliserons gRPC qui repose sur HTTP/2.

- Multiplexage des échanges sur une même connexion
- Compression des headers des requêtes (optimisation)
- Une meilleure utilisation des connexions persistantes.

$\Rightarrow$ Transmission efficace des flux de données

Un peu d'histoire : des RPC à gRPC

RPC dans les années 80

Le contexte

Les stations de travail et les réseaux locaux se développent : les programmes doivent **partager des services entre machines**.

RPC dans les années 80

Le contexte

Les stations de travail et les réseaux locaux se développent : les programmes doivent **partager des services entre machines**.

- **Début des années 1980 :** le modèle RPC est formalisé pour les systèmes distribués.

RPC dans les années 80

Le contexte

Les stations de travail et les réseaux locaux se développent : les programmes doivent **partager des services entre machines**.

- **Début des années 1980 :** le modèle RPC est formalisé pour les systèmes distribués.
- **Milieu des années 1980 : Sun RPC** (Sun Microsystems) se diffuse dans le monde Unix. Il est notamment utilisé par **NFS** pour accéder à des fichiers distants.

RPC dans les années 80

Le contexte

Les stations de travail et les réseaux locaux se développent : les programmes doivent **partager des services entre machines**.

- **Début des années 1980 :** le modèle RPC est formalisé pour les systèmes distribués.
- **Milieu des années 1980 : Sun RPC** (Sun Microsystems) se diffuse dans le monde Unix. Il est notamment utilisé par **NFS** pour accéder à des fichiers distants.
- **La difficulté suivante :** faire coopérer des applications construites dans des environnements différents.

La suite de l'histoire

Passer du partage de services à **l'interopérabilité**.

CORBA

- **1991 :** l'OMG standardise **CORBA** (*Common Object Request Broker Architecture*). But : faire coopérer des **objets distribués** malgré des langages, systèmes et architectures différents.

Application Java

Application C++

CORBA

- **1991 :** l'OMG standardise **CORBA** (*Common Object Request Broker Architecture*). But : faire coopérer des **objets distribués** malgré des langages, systèmes et architectures différents.
- Un **ORB** (*Object Request Broker*) permet d'accéder aux objets distants ; leurs interfaces sont définies en IDL.



CORBA

- **1991 :** l'OMG standardise **CORBA** (*Common Object Request Broker Architecture*). But : faire coopérer des **objets distribués** malgré des langages, systèmes et architectures différents.
- Un **ORB** (*Object Request Broker*) permet d'accéder aux objets distants ; leurs interfaces sont définies en IDL.
- Les ORB communiquent notamment avec **IIOP**, généralement au-dessus de TCP/IP.



CORBA

- **1991 :** l'OMG standardise **CORBA** (*Common Object Request Broker Architecture*). But : faire coopérer des **objets distribués** malgré des langages, systèmes et architectures différents.
- Un **ORB** (*Object Request Broker*) permet d'accéder aux objets distants ; leurs interfaces sont définies en IDL.
- Les ORB communiquent notamment avec **IIOP**, généralement au-dessus de TCP/IP.



Limite

Une solution ambitieuse, mais **complexe à déployer et à administrer**.

XML-RPC

Fin des années 1990 : les RPC rencontrent le Web

Plutôt que de développer une infrastructure réseau spécifique, **XML-RPC** s'appuie sur **HTTP** pour le transport et **XML** pour représenter les appels.

XML-RPC

Fin des années 1990 : les RPC rencontrent le Web

Plutôt que de développer une infrastructure réseau spécifique, **XML-RPC** s'appuie sur **HTTP** pour le transport et **XML** pour représenter les appels.

Ce que cela apporte

- Réutilisation de l'infrastructure du Web.
- Échanges compréhensibles entre technologies différentes.
- Mise en œuvre relativement simple.

XML-RPC

Fin des années 1990 : les RPC rencontrent le Web

Plutôt que de développer une infrastructure réseau spécifique, **XML-RPC** s'appuie sur **HTTP** pour le transport et **XML** pour représenter les appels.

Ce que cela apporte

- Réutilisation de l'infrastructure du Web.
- Échanges compréhensibles entre technologies différentes.
- Mise en œuvre relativement simple.

Ce qui manque encore

- Messages XML verbeux.
- Modèle de types et fonctionnalités limités.
- Peu de standards complémentaires.

XML-RPC

Fin des années 1990 : les RPC rencontrent le Web

Plutôt que de développer une infrastructure réseau spécifique, **XML-RPC** s'appuie sur **HTTP** pour le transport et **XML** pour représenter les appels.

Ce que cela apporte

- Réutilisation de l'infrastructure du Web.
- Échanges compréhensibles entre technologies différentes.
- Mise en œuvre relativement simple.

Ce qui manque encore

- Messages XML verbeux.
- Modèle de types et fonctionnalités limités.
- Peu de standards complémentaires.

Étape suivante : structurer et standardiser les Web Services.

SOAP

- **À partir de 1998 : SOAP** (*Simple Object Access Protocol*) vise des **Web Services fortement standardisés**.

SOAP

- **À partir de 1998 : SOAP** (*Simple Object Access Protocol*) vise des **Web Services fortement standardisés**.
- Les messages XML prennent une structure commune : une **enveloppe**, avec un **en-tête** optionnel et un **corps**.

Schéma d'un message SOAP

```
Envelope { Header (optionnel) | Body }
```

SOAP

- **À partir de 1998 : SOAP** (*Simple Object Access Protocol*) vise des **Web Services fortement standardisés**.
- Les messages XML prennent une structure commune : une **enveloppe**, avec un **en-tête** optionnel et un **corps**.
- Autour de SOAP se développe un écosystème : **WSDL** pour décrire les services, et des standards **WS-*** pour des besoins tels que la sécurité ou la fiabilité.

Schéma d'un message SOAP

```
Envelope { Header (optionnel) | Body }
```

SOAP

- **À partir de 1998 : SOAP** (*Simple Object Access Protocol*) vise des **Web Services fortement standardisés**.
- Les messages XML prennent une structure commune : une **enveloppe**, avec un **en-tête** optionnel et un **corps**.
- Autour de SOAP se développe un écosystème : **WSDL** pour décrire les services, et des standards **WS-*** pour des besoins tels que la sécurité ou la fiabilité.

Schéma d'un message SOAP

```
Envelope { Header (optionnel) | Body }
```

Contrepartie

Un cadre riche, utile notamment dans certains systèmes d'entreprise, mais une **pile de standards complexe** et des échanges XML coûteux.

REST

2000 : une autre manière de concevoir les services

Roy Fielding formalise **REST**, un style architectural centré sur les **ressources du Web**, plutôt que sur les procédures distantes.

REST

2000 : une autre manière de concevoir les services

Roy Fielding formalise **REST**, un style architectural centré sur les **ressources du Web**, plutôt que sur les procédures distantes.

Approche RPC

```
getUser(42)
```

On invoque une **opération**.

Approche REST

```
GET /users/42
```

On accède à une **ressource**.

REST

2000 : une autre manière de concevoir les services

Roy Fielding formalise **REST**, un style architectural centré sur les **ressources du Web**, plutôt que sur les procédures distantes.

Approche RPC

`getUser(42)`

On invoque une **opération**.

Approche REST

`GET /users/42`

On accède à une **ressource**.

- REST se généralise avec les **API Web publiques**, notamment grâce à sa simplicité d'intégration.

REST

2000 : une autre manière de concevoir les services

Roy Fielding formalise **REST**, un style architectural centré sur les **ressources du Web**, plutôt que sur les procédures distantes.

Approche RPC

```
getUser(42)
```

On invoque une **opération**.

Approche REST

```
GET /users/42
```

On accède à une **ressource**.

- REST se généralise avec les **API Web publiques**, notamment grâce à sa simplicité d'intégration.
- Mais les RPC ne disparaissent pas : les échanges **entre services** posent d'autres contraintes de typage, de volume et de fréquence des appels.

Protocol Buffers

Années 2000 : Google travaille sur le coût des échanges

Protocol Buffers (Protobuf) propose une **sérialisation binaire compacte**, développée avant gRPC.

Protocol Buffers

Années 2000 : Google travaille sur le coût des échanges

Protocol Buffers (Protobuf) propose une **sérialisation binaire compacte**, développée avant gRPC.

- Là où XML et JSON transportent des noms de champs en texte, Protobuf encode les champs à l'aide de **numéros** et de valeurs binaires.

Protocol Buffers

Années 2000 : Google travaille sur le coût des échanges

Protocol Buffers (Protobuf) propose une **sérialisation binaire compacte**, développée avant gRPC.

- Là où XML et JSON transportent des noms de champs en texte, Protobuf encode les champs à l'aide de **numéros** et de valeurs binaires.
- Les messages peuvent **évoluer progressivement** (par exemple en ajoutant des champs), à condition de respecter les règles de compatibilité du format.

Protocol Buffers

Années 2000 : Google travaille sur le coût des échanges

Protocol Buffers (Protobuf) propose une **sérialisation binaire compacte**, développée avant gRPC.

- Là où XML et JSON transportent des noms de champs en texte, Protobuf encode les champs à l'aide de **numéros** et de valeurs binaires.
- Les messages peuvent **évoluer progressivement** (par exemple en ajoutant des champs), à condition de respecter les règles de compatibilité du format.
- Protobuf est **indépendant du transport** : il peut servir à échanger ou à stocker des données sans utiliser gRPC.

La brique qui manquait

Un format efficace pour les échanges entre services, que gRPC adoptera comme **format par défaut**.

2015 : Google publie gRPC en Open Source

Une réponse aux besoins des **architectures distribuées modernes**, notamment des applications composées de nombreux microservices.

2015 : Google publie gRPC en Open Source

Une réponse aux besoins des **architectures distribuées modernes**, notamment des applications composées de nombreux microservices.

- gRPC réunit des briques déjà rencontrées : **RPC**, **contrats Protobuf** et **HTTP/2**.

2015 : Google publie gRPC en Open Source

Une réponse aux besoins des **architectures distribuées modernes**, notamment des applications composées de nombreux microservices.

- gRPC réunit des briques déjà rencontrées : **RPC**, **contrats Protobuf** et **HTTP/2**.
- Il ajoute un cadre pratique pour les communications **multi-langages**, avec des échanges classiques ou des **flux continus** dans les deux sens.

2015 : Google publie gRPC en Open Source

Une réponse aux besoins des **architectures distribuées modernes**, notamment des applications composées de nombreux microservices.

- gRPC réunit des briques déjà rencontrées : **RPC**, **contrats Protobuf** et **HTTP/2**.
- Il ajoute un cadre pratique pour les communications **multi-langages**, avec des échanges classiques ou des **flux continus** dans les deux sens.

REST et gRPC peuvent coexister



gRPC

Un framework RPC moderne

gRPC : framework **Open Source**, initialement développé par Google et publié en 2015.

- Conçu pour les communications **entre services**, notamment dans les architectures distribuées et microservices.
- Repose sur **HTTP/2** et utilise **Protocol Buffers** par défaut.
- Prend en charge de nombreux langages : **Java, Python, Go, C#, C++, JavaScript**, etc.
- Utilisé dans de grands écosystèmes de services : Google, Netflix, Cisco, etc.

Documentation : `https://grpc.io/docs/languages/`

IDL de gRPC - protocol buffers

Protocol Buffers (Protobuf)

Un **langage de définition** des messages/services (fichiers `.proto`) et un **format binaire** de sérialisation.

- Un `message` décrit les données échangées : champs, types, identifiants numériques.
- Un `service` regroupe les procédures RPC et leurs types de requêtes/réponses.
- Le compilateur `protoc` pour **générer du code**

Documentation : `https://protobuf.dev/`

IDL de gRPC - protocol buffers

Structure typique d'un fichier `user_service.proto`

```
1  syntax = "proto3";
2  package course.users;
3  option java_package = "fr.univ.m1miage.grpc";
4  option java_multiple_files = true;
5
6  message UserRequest { ... }
7  message UserDTO     { ... }
8  message ImportResult { ... }
9  message Empty {}
10
11 service UserService {
12   rpc GetUser(UserRequest) returns (UserDTO);
13   rpc GetUsers(Empty) returns (stream UserDTO);
14   rpc ImportUsers(stream UserDTO) returns (ImportResult);
15   rpc EchoUsers(stream UserDTO) returns (stream UserDTO);
16 }
```

Les types de données

Type Protobuf	Rôle
`int32`, `int64`	Entiers
`sint32`, `sint64`	Entiers signés optimisés
`float`, `double`	Réels
`bool`	Booléen
`string`	Texte UTF-8
`bytes`	Données binaires
`enum`, `message`	Types composés

```
1 message UserRequest {
2     int32 id = 1;
3 }
```

- `id` : nom du champ.
- `int32` : type.
- `1` : numéro du champ **sur le fil**.

Numérotation

Chaque champ possède un numéro **unique et stable**. Ne jamais réutiliser un numéro supprimé : utiliser `reserved`.

Des exemples : Message simple

Encodage d'un `UserDTO`

```
message UserDTO {
  int32  id     = 1;
  string name   = 2;
  string email  = 3;
  bool   active = 4;
}
```

- Quatre champs typés, identifiés par les numéros 1, 2, 3, 4.
- Le message joue le rôle d'un **DTO** dans une application Java.
- Les classes Java correspondantes seront **générées**, sans écrire de sérialiseur à la main.

Des exemples : Tableaux et listes

Utilisation de `repeated`

```proto
message UserList {
  repeated UserDTO users = 1;
}

message Statistics {
  repeated int32 values = 1;
}
```

- `repeated` : **zéro, un ou plusieurs** éléments du même type.
- Côté Java, les méthodes générées sont adaptées aux listes (accesseurs supplémentaires : `addUsers()`, `getUsersList()`, etc.).

`repeated` ≠ `stream`

Ne pas confondre `repeated UserDTO` (une liste dans un message) et `stream UserDTO` (plusieurs messages dans une RPC).

Des exemples : Messages imbriqués et composition

```
message UserProfile {
  message Address {
    string city = 1;
    string street = 2;
  }
  UserDTO user = 1;
  Address home = 2;
}
```

- Un champ peut référencer **un autre message**.
- Un type peut être déclaré **à l'intérieur** d'un autre message.
- En Java, le type imbriqué s'utilise comme une classe interne (par ex. `UserProfile.Address`).

Les types de procédures

4 types de procédures

- **Unary RPC** : une requête, une réponse
- **Server streaming** : une requête, plusieurs réponses
- **Client streaming** : plusieurs requêtes, une réponse
- **Bidirectional streaming** : plusieurs requêtes, plusieurs réponses (deux flux indépendants)

Les types de procédures : Unary RPC

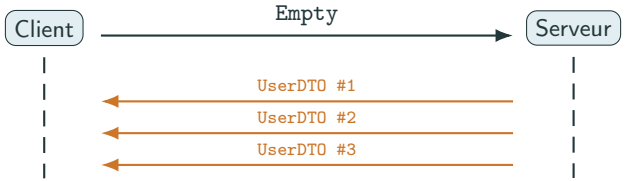
```
1 rpc GetUser(UserRequest) returns (UserDTO);
```



- Semblable à un appel de fonction classique.
- Exemple : obtenir le profil de l'utilisateur 42.

Les types de procédures : Server streaming

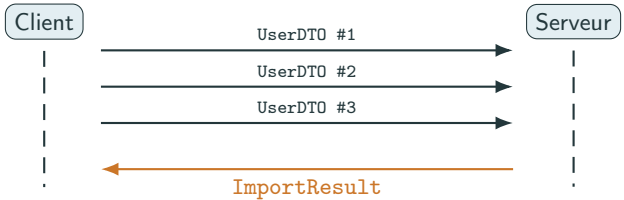
```
1 rpc GetUsers(Empty) returns (stream UserDTO);
```



- Le client fait **un seul appel**.
- Le serveur renvoie une **suite de messages**, puis termine le flux.

Les types de procédures : Client streaming

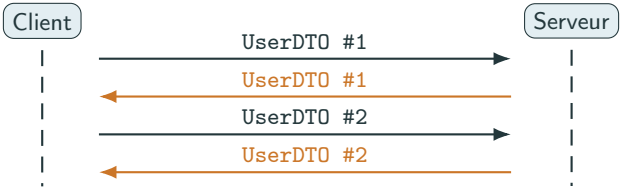
```
1 rpc ImportUsers(stream UserDTO) returns (ImportResult);
```



- Le client envoie des **messages successifs**.
- Le serveur renvoie un seul message.

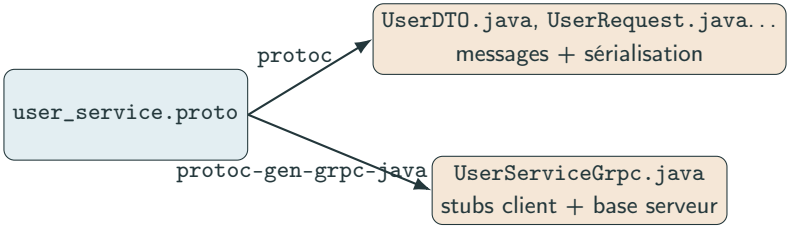
Les types de procédures : Bidirectional streaming

```
1 rpc EchoUsers(stream UserDTO) returns (stream UserDTO);
```



- Les deux côtés peuvent **envoyer et recevoir** au fil de l'eau.
- Les flux sont **indépendants** : pas nécessairement un aller-retour à chaque message.

Compilation du fichier `proto`



Generation des fichiers dans un projet Maven (plugins Protobuf/gRPC configurés)

```
./mvnw generate-sources
```

Fichiers générés : Les stubs

Client : `UserServiceGrpc`

Note pour plus tard :

- `UserServiceBlockingStub`
- `UserServiceStub` (asynchrone)
- `UserServiceFutureStub` (unary)

Le client **crée un stub** qui sera lié à un **Channel**, pour invoquer ses méthodes.

Serveur : `UserServiceImplBase`

- Classe de base générée.
- L'implémentation doit **hériter** de cette classe.
- Elle **définit** les procédures exposées dans le `.proto`.

Ne **jamais modifier** les fichiers générés : ils sont régénérés à partir du contrat `.proto`.

Implementation coté serveur

Spring Boot + Spring gRPC

`@GrpcService` enregistre l'implémentation auprès du serveur gRPC, comme `@RestController` expose un contrôleur HTTP.

```
1 @GrpcService
2 public class UserService
3         extends UserServiceGrpc.UserServiceImplBase {
4     // Redefinition des methodes du fichier .proto
5 }
```

- `UserServiceImplBase` est **générée** : on hérite de son contrat.
- Le service gRPC peut déléguer à la **même couche métier** qu'une API REST.
- Le framework Spring fournit le transport et l'enregistrement du service.

Implementation coté serveur : Unary RPC

```
1 rpc GetUser(UserRequest) returns (UserDTO);
```

Signature Java à redéfinir :

```
1 @Override
2 public void getUser(
3     UserRequest request,
4     StreamObserver<UserDTO> responseObserver) {
5     // ...
6 }
```

- **Un** message entrant, **un** message sortant.
- `StreamObserver` **n'implique pas** une RPC streaming : c'est aussi l'API de réponse des Unary RPC en Java.

Implementation coté serveur : Server streaming

```
1 rpc GetUsers(Empty) returns (stream UserDTO);
```

Signature Java à redéfinir :

```
1 @Override
2 public void getUsers(
3     Empty request,
4     StreamObserver<UserDTO> responseObserver) {
5     // ...
6 }
```

- Même **signature Java** que pour une Unary RPC.
- Différence : le serveur peut appeler `onNext()` **plusieurs fois** avant la terminaison.

Implementation coté serveur : Client streaming

```
1 rpc ImportUsers(stream UserDTO) returns (ImportResult);
```

Signature Java à redéfinir :

```
1 @Override
2 public StreamObserver<UserDTO> importUsers(
3     StreamObserver<ImportResult> responseObserver) {
4     // Retourner un observer pour recevoir les UserDTO
5 }
```

- L'**argument** est l'observer des réponses envoyées au client.
- La **valeur retournée** est l'observer dans lequel arriveront les requêtes du client.

Implementation coté serveur : Bidirectional streaming

```
rpc EchoUsers(stream UserDTO) returns (stream UserDTO);
```

Signature Java à redéfinir :

```
@Override
public StreamObserver<UserDTO> echoUsers(
    StreamObserver<UserDTO> responseObserver) {
    // Retourner l'observer des requetes entrantes
}
```

- Signature proche du client streaming.
- La réponse peut être produite **au fil de la réception**, avant même que le client ait terminé ses envois.

Construction des messages

Les messages Java Protobuf sont immuables

Création grâce au **Builder** généré : `newBuilder()` → `set...()` → `build()`.

```java
UserDTO user = UserDTO.newBuilder()
    .setId(42)
    .setName("Alice")
    .setEmail("alice@example.org")
    .setActive(true)
    .build();
```

- Même principe **côté client** (requêtes) et **côté serveur** (réponses).
- Pour les listes : `addUsers(user)` ou `addAllUsers(users)`.
- Dans une application : utiliser un **mapper** `Entité métier` ↔ `message Protobuf`.

Implémentation côté serveur

```
responseObserver.onNext(message);          // Emission
responseObserver.onCompleted();            // Succes
// OU : responseObserver.onError(error); // Echec
```

- `onNext(message)` transmet une réponse (ou un élément du flux).
- `onCompleted()` **termine normalement** l'appel.
- `onError(Throwable)` **termine en erreur** l'appel.
- **Une seule terminaison** : `onCompleted()` *ou* `onError()`, jamais les deux.

Cardinalité

Unary : au plus une émission ; server streaming : plusieurs possibles. Les flux entrants ont leur propre `StreamObserver` pour les RPC client/bidirectionnelles.

Implémentation côté serveur : Unary RPC

```
rpc GetUser(UserRequest) returns (UserDTO);
```

```
@Override
public void getUser(UserRequest request,
        StreamObserver<UserDTO> responseObserver) {
    UserDTO user = UserDTO.newBuilder()
        .setId(request.getId())
        .setName("Alice")
        .setEmail("alice@example.org")
        .setActive(true)
        .build();

    responseObserver.onNext(user);
    responseObserver.onCompleted();
}
```

Implémentation côté serveur : Server streaming

```
rpc GetUsers(Empty) returns (stream UserDTO);
```

```
@Override
public void getUsers(Empty request,
        StreamObserver<UserDTO> responseObserver) {

    UserDTO alice = UserDTO.newBuilder().setId(1).setName("Alice").build();
    UserDTO bob = UserDTO.newBuilder().setId(2).setName("Bob").build();

    responseObserver.onNext(alice);
    responseObserver.onNext(bob);
    responseObserver.onCompleted();
}
```

Idée : les messages sont reçus séparément par le client, sans être encapsulés dans un `repeated UserDTO`.

Implémentation côté serveur : Client streaming

```
rpc ImportUsers(stream UserDTO) returns (ImportResult);
```

```
@Override
public StreamObserver<UserDTO> importUsers(
        StreamObserver<ImportResult> responseObserver) {
    return new StreamObserver<UserDTO>() {
        private int count = 0;
        @Override public void onNext(UserDTO user) {
            count++; // Ici : sauvegarder user
        }
        @Override public void onError(Throwable t) {
            // Journaliser l'interruption du flux entrant
        }
        @Override public void onCompleted() {
            responseObserver.onNext(ImportResult.newBuilder()
                .setCount(count).build());
            responseObserver.onCompleted();
        }
    };
}
```

Implémentation côté serveur : Bidirectional streaming

```
rpc EchoUsers(stream UserDTO) returns (stream UserDTO);
```

```
@Override
public StreamObserver<UserDTO> echoUsers(
        StreamObserver<UserDTO> responseObserver) {
    return new StreamObserver<UserDTO>() {
        @Override public void onNext(UserDTO user) {
            responseObserver.onNext(user); // Echo
        }
        @Override public void onError(Throwable t) {
            // Journaliser l'interruption
        }
        @Override public void onCompleted() {
            responseObserver.onCompleted();
        }
    };
}
```

Remarque : les deux flux peuvent aussi fonctionner de manière indépendante ; cet écho est seulement un exemple facile à lire.

Implémentation côté serveur : Erreur

Exemple : identifiant de requête invalide

```java
@Override
public void getUser(UserRequest request,
        StreamObserver<UserDTO> responseObserver) {
    if (request.getId() <= 0) {
        responseObserver.onError(
            Status.INVALID_ARGUMENT
                .withDescription("Id invalide")
                .asRuntimeException());
        return;
    }
    // ... chercher l'utilisateur, sinon NOT_FOUND
}
```

- Une erreur gRPC est transmise avec un **status** et éventuellement une description.
- `onError()` clôt l'appel (comme `onCompleted()`).

Erreurs standards

Erreur gRPC	Situation habituelle	Analogie HTTP
`INVALID_ARGUMENT`	Paramètre invalide	400
`NOT_FOUND`	Ressource absente	404
`ALREADY_EXISTS`	Création en conflit	409
`PERMISSION_DENIED`	Droits insuffisants	403
`UNAUTHENTICATED`	Identité non vérifiée	401
`INTERNAL`	Erreur interne	500
`UNAVAILABLE`	Service indisponible	503

Attention

Ce sont des **analogies sémantiques**, **pas** une correspondance exacte des statuts sur le fil. gRPC possède son propre code de statut.

Appel d'une procédure distante

Côté client : trois étapes

1. Créer ou obtenir un **Channel** vers le serveur.
2. Créer un **stub** généré associé au Channel.
3. Appeler une **méthode distante** à travers le stub.



Le Channel représente la connexion logique et se réutilise pour plusieurs appels.

Appel distant : 1 - Channel côté client

```
ManagedChannel channel = ManagedChannelBuilder
    .forAddress("localhost", 9090)
    .usePlaintext()
    .build();

UserServiceGrpc.UserServiceBlockingStub stub =
    UserServiceGrpc.newBlockingStub(channel);
```

- Un Channel peut être **partagé et réutilisé** entre appels.
- Fermer proprement le Channel lors de l'arrêt du client : `channel.shutdown()`.

Appel distant : 1 - Channel côté client

Definir des channels par defaut dans `application.properties`

```
spring.grpc.client.channels.users.address=localhost:9090
```

Créer le Channel avec `GrpcChannelFactory`

```java
@Autowired
private GrpcChannelFactory channelFactory;

public void initClient() {
    ManagedChannel channel =
        channelFactory.createChannel("users");
    // Utiliser channel pour creer le stub
}
```

Appel distant : 2 - Créer un stub

Les variantes générées par `UserServiceGrpc`

```
1 var blocking = UserServiceGrpc.newBlockingStub(channel);
2 var async    = UserServiceGrpc.newStub(channel);
3 var future   = UserServiceGrpc.newFutureStub(channel);
```

- **Blocking** : unary `Response` en retour direct ; server streaming par `Iterator<Response>`.
- **Async** : `StreamObserver`, pour **les quatre formes** de RPC, sans bloquer l'appelant.
- **Future** : retourne une `ListenableFuture<Response>` pour les **Unary RPC**.

Choix pratique

Recommendation : `BlockingStub` sur les RPC simples ; `Stub` asynchrone pour envoyer un flux ou échanger dans les deux sens.

Appel distant : 3 - Appeler une méthode Unary RPC

```
rpc GetUser(UserRequest) returns (UserDTO);
```

```
UserRequest req = UserRequest
    .newBuilder().setId(42).build();

UserDTO user = stub.getUser(req);
```

Appel distant : 3 - Appeler une méthode Server streaming

```
rpc GetUsers(Empty) returns (stream UserDTO);
```

```
Iterator<UserDTO> it = stub
    .getUsers(Empty.getDefaultInstance());

while(it.hasNext()){
    UserDTO user = it.next();
    System.out.println(user.getName())
}
```

Appel distant : 3 - Appeler une méthode Client streaming

```
rpc ImportUsers(stream UserDTO) returns (ImportResult);
```

```
StreamObserver<UserDTO> input =
    stub.importUsers(resultObserver);

input.onNext(alice);
input.onNext(bob);
input.onCompleted();
```

Appel distant : 3 - Appeler une méthode Bidirectional streaming

```
rpc EchoUsers(stream UserDTO) returns (stream UserDTO);
```

```
StreamObserver<UserDTO> input =
    stub.echoUsers(replyObserver);
input.onNext(alice);

// replyObserver recoit les reponses
input.onCompleted();
```